Changelog

[Unreleased] — 2026-03-07

Fixed

- **`detect_swing_windows_from_imu()` in `emg_util.py`**: Added `already_in_dps: bool = True` parameter (default `True` — backward-compatible with all existing callers that pass dps data from `load_session_stitched()`).
- `True` (default): gx/gy/gz already in dps — no FSR conversion. Standard path for `load_session_stitched()` data (yaml_runner, tests, direct callers).
- `False`: raw LSB values — converts using `gyro_fsr_dps / acc_fsr_g` columns. Opt-in for callers that pass raw ADC data.
- **Note:** initial implementation used `False` as default, which broke `test_detect_swings_from_imu` (0 swings detected because dps values were divided by the scale factor again). Corrected to `True` to match pre-change behavior.
- **`detect_swings_from_imu()` in `emg_util_yaml_runner.py`**: Replaced ~70-line duplicate implementation with a thin wrapper that delegates to `core.detect_swing_windows_from_imu(already_in_dps=True)`. The previous local copy:
- Was missing the window-merge step (merging overlapping `[t_start, t_end]` windows)
- Used a simple monotonicity fix for `t_rel_s` instead of `build_time_axis_from_ts()`
- Duplicated the peak-finding and min-gap enforcement logic verbatim The yaml_runner now gets the window-merge step for free. No behavior change for current parameters (window merge is a no-op when `pre_ms + post_ms < min_gap_ms`).

Updated

- **`HIGH_LEVEL_DESIGN.md`**:
- "Offline path" description: updated from "robust z-score" to "fixed 300 dps threshold"
- "Swing Detection: Real-time vs Offline" table: expanded to 3 columns (Android / PC plotter / offline), corrected thresholds (300 dps for plotter + offline), corrected debounce (2000ms), added `already_in_dps` implementation note, added window-merge column
- Added known misalignment note: Android Java still uses ~150 dps / 300ms (TODO to fix)
- "FatigueMonitor: Canonical Implementation" section: updated to reflect `fatigue_monitor.py` as the single source of truth (no longer duplicated between `emg_util.py` and `emg_util_nopandas.py`)
- "Key Files" table: added `fatigue_monitor.py` entry

- "Known Issues": removed stale #1 (FatigueMonitor duplication — fixed); renumbered; added new #1 (Android threshold misalignment) and new #5 (offline vs real-time baseline RMS filter difference)

All notable changes to `emg_util` are documented here.
Format follows Keep a Changelog.

[Unreleased]

[2026-03-07] — Fix: `FatigueMonitor` baseline missing RMS filter (`fatigue_monitor.py`)

Root cause

`FatigueMonitor.push_swing()` collected the first `baseline_swings` (default 20) post-warmup swings and used their median MDF/RMS as the baseline — with no check for noise-dominated swings. If the first 20 swings had poor electrode contact (RMS near noise floor, MDF ≈ 100 Hz flat spectrum), the baseline was anchored at ≈ 100 Hz. Every subsequent real swing (MDF ≈ 160–200 Hz) then computed `(100 - 200) / 100 × 100 = -100%` — the opposite of fatigue.

This is the same bug that was fixed in `emg_util_yaml_runner.py` via `_filter_baseline_by_rms()` on 2026-03-05. `FatigueMonitor` (used by `sensordata_plotter.py --fatigue` and Android) had the same gap.

Fixed (`fatigue_monitor.py`)

- In `push_swing()`, replaced the simple "collect N, finalize" logic with a **real-time adaptive RMS filter**:
- Collect candidates continuously after warmup.
- After each new candidate (once `n_collected >= baseline_swings`), compute `p90_rms` of all collected candidates and apply `rms_thr = 0.25 × p90_rms`.
- **Finalize when** either:
  - `n_valid >= baseline_swings` (enough good candidates found), OR
  - `n_collected >= baseline_swings × 3` (hard cap — fall back to best available)
- If ≥ 3 valid candidates, use filtered set; otherwise fall back to all.
- Use `np.median()` of the surviving set for both `baseline_mdf` and `baseline_rms`.
- Prints diagnostic: `[FatigueMonitor] Baseline: N_valid/N_collected valid ...`
- **Why 3× hard cap works in real-time:** if the first `baseline_swings` candidates are all noise-dominated (poor electrode contact), the p90 of those N is still noise-level and the filter does nothing. By collecting up to 3× candidates, electrode contact typically improves and later

candidates raise the p90, allowing the filter to correctly exclude the early noise-dominated swings.

- **Example (default baseline_swings=20):**
- Swings 1–20: noise-dominated (MDF≈100 Hz, low RMS) → p90 is noise-level → 0 valid → keep collecting
- Swings 21–40: electrode contact improves → p90 rises → early swings filtered out → ≥20 valid → finalize
- Hard cap at swing 60: if contact never improves, use best available
- Mirrors `_filter_baseline_by_rms()` in `emg_util_yaml_runner.py` in principle, adapted for streaming (no session-wide p90 available in real-time).
- Android uses `emg_util` as a git submodule — no manual copy needed.

Affected paths

- `sensordata_plotter.py --mode offline --fatigue` — uses `FatigueMonitor` directly
- `sensordata_plotter.py --mode live --fatigue` — same
- Android `analysis_bridge.py` — same via Chaquopy

[2026-03-07] — Fix: `analyze_tennis_session()` FSR defaults wrong (`emg_util_yaml_runner.py`)

Root cause

`analyze_tennis_session()` called `loader.load_session_stitched()` with hardcoded fallback defaults `acc_fsr_default_g=2.0` and `gyro_fsr_default_dps=250.0` — the **old pre-2026-02-25 values**. The 2026-02-25 FSR update correctly changed these defaults in `capture_loader.py`, `config.yaml`, `config_fast.yaml`, and `analyze_single_session()`, but missed `analyze_tennis_session()` (the main path).

Impact: Without an explicit config file, gyro is scaled by 131.0 (250 dps FSR) instead of 32.8 (1000 dps FSR) → gyro values 4× too small → a real 300 dps swing appears as ~75 dps → zero swings detected.

Fixed (`emg_util_yaml_runner.py`)

- `analyze_tennis_session()`: changed `acc_fsr_default_g=2.0` → `4.0` and `gyro_fsr_default_dps=250.0` → `1000.0` to match `config.yaml` and firmware.
- Added comment explaining why the correct defaults matter.

[2026-03-07] — Fix: `fatigue_summary.csv` baseline inconsistent with `baseline.json` (`emg_util_yaml_runner.py`)

Root cause

`_compute_per_swing_fatigue_summary()` recomputed the baseline MDF independently from `baseline.json`, using the **mean of the first 10% of post-warmup swings with no RMS filter**. Meanwhile `baseline.json` uses the **median of the first 20 post-warmup swings after filtering out noise-dominated (low-RMS) swings**.

Result: `fatigue_summary.csv` reported `baseline_MDF = 137.1 Hz` while `baseline.json` reported `baseline.mdf = 166.0 Hz` for the same session. `swing_features.csv` (`swing_mdf_drop_pct`) was computed relative to 166 Hz, making the two files inconsistent and incomparable.

Fixed (`emg_util_yaml_runner.py`)

`_compute_per_swing_fatigue_summary()` — new `ext_baseline_mdf` parameter

- Added `ext_baseline_mdf: float = None` parameter.
- When provided and finite/positive, uses it directly as `baseline_mdf` (`baseline_source = 'baseline_json'`).
- Falls back to mean of first 10% of post-warmup swings when `None` (`baseline_source = 'first_10pct_mean'`).
- Added `baseline_source` field to `fatigue_summary.csv` output row.
- Fixed latent bug: `baseline_n` is now always computed (needed for MNF computation even when `ext_baseline_mdf` is provided for MDF).

Call sites updated — both medium mode and full mode

- Medium mode: `_compute_per_swing_fatigue_summary(..., ext_baseline_mdf=base.get('mdf'))`
- Full mode: `_compute_per_swing_fatigue_summary(..., ext_baseline_mdf=base.get('mdf'))`

Expected behavior after fix

`fatigue_summary.csv` `baseline_MDF` now matches `baseline.json` `baseline.mdf` exactly. `MDF_drop_pct` in `fatigue_summary.csv` is now consistent with `swing_mdf_drop_pct` in `swing_features.csv`.

[2026-03-05] — Fix: fatigue summary -53% nonsensical result (`emg_util_yaml_runner.py`)

Root cause

`core.analyze_fatigue(res.feats, ...)` used 1-second windowed features that include rest periods between rallies. During rest the EMG signal is noise-dominated (flat spectrum, MDF ≈ 100 Hz). The windowed MDF at session end (more active play) was higher than at session start (more rest), giving `(100 - 254) / 100 × 100 = -53%` — the opposite of fatigue.

Fixed (`emg_util_yaml_runner.py`)

Bug — windowed MDF corrupts fatigue summary

- Added `_compute_per_swing_fatigue_summary(swing_feat_df, out_dir, warmup_skip_ms)`:
- Filters out pre-warmup swings (`t_peak_ms < warmup_skip_ms`).
- Baseline MDF = mean of first 10% of post-warmup swings.
- Final MDF = mean of last 10% of post-warmup swings.
- Computes `MDF_drop_pct`, `MNF_drop_pct` (if available), fatigue onset time.
- Saves `fatigue_summary.csv` with `source = per_swing`.
- Prints `[Fatigue/per-swing] MDF drop % = X.XX ; MNF drop % = Y.YY ; onset_s = Z`.
- Called in both **medium mode** and **full mode** after `_plot_fatigue_trend_stitched()`.
- The `do_fatigue` block in full mode now has `pass` — `core.analyze_fatigue` (windowed) is no longer called, preventing it from overwriting `fatigue_summary.csv` with noise-contaminated results.

Also fixed — f-string format spec error

- `f'{onset_time:.1f if np.isfinite(onset_time) else "n/a"}'` is invalid Python (ternary inside format spec). Fixed to pre-compute `onset_str` before the f-string.

Verified output (`out_0305_v3/fatigue_summary.csv`)

Field	Value	Interpretation
`baseline_MDF`	137.1 Hz	Real muscle activity (not noise)
`final_MDF`	140.7 Hz	Slightly higher at end
`MDF_drop_pct`	**−2.6%**	Near zero — no significant fatigue ✓
`MNF_drop_pct`	+3.9%	Consistent with MDF
`fatigue_onset_s`	841.6 s	No onset (MDF never dropped 15% below baseline)
`n_swings_post_warmup`	505	505 swings after 10-min warmup exclusion
`source`	`per_swing`	Swing-gated, not rest-period contaminated

[2026-03-05] — Fix: fatigue trend plot nonsensical results (`emg_util_yaml_runner.py`)

Root-cause analysis

The `fatigue_trend.png` plot showed two symptoms that made the analysis uninterpretable:

1. **MDF drop % deeply negative (−50 % to −100 %) throughout the entire session.**

- Expected: near 0 % at session start, going positive (e.g. +12–20 %) as the player fatigues.
- Observed: large negative values from the very first swing, implying MDF *increased* over time.
- **Root cause:** The first BLE connection (≈ t = 0–500 s) had near-zero EMG RMS — the electrode was not yet in good contact with the skin. The IMU-based swing detector still fired during this period (gyro threshold is independent of EMG quality). Those swing windows contained mostly broadband noise, whose MDF ≈ 100 Hz (flat spectrum). `select_baseline_swings()` picked the first N active swings after the warmup skip, which still fell inside this low-RMS period. The baseline MDF was therefore anchored at ≈ 100 Hz. Every subsequent swing (real muscle activity, MDF ≈ 160–200 Hz) then computed `(100 − 200) / 100 × 100 = −100 %` — the opposite of fatigue.

1. **Acc drop % reference lines labelled `"mild (-10%)" / "significant (-20%)"` but drawn at +10 and +20.**

- The sign in the legend strings was inverted relative to the actual line positions.

Fixed (`emg_util_yaml_runner.py`)

Bug 1 — Wrong acc drop % legend labels

- In `_plot_fatigue_trend_stitched()`, changed:
- `label='mild (-10%)'` → `label='mild (+10%)'`
- `label='significant (-20%)'` → `label='significant (+20%)'`
- Labels now match the drawn line positions (+10 % and +20 %).

Bug 2 — Noise-dominated baseline corrupts MDF drop %

- Added `_filter_baseline_by_rms(base_feats, all_swing_feats, out_dir)` helper:
- Computes the **90th percentile RMS** across all swings to represent "peak effort".
- The median RMS is not robust because a session might be >50% inactive/noise swings, which would pull the median down to the noise floor.
- Drops any baseline candidate whose `rms < 0.25 × session_p90_rms` (noise-dominated).
- Falls back to a 10% threshold, or the original candidate set, if fewer than 3 swings survive the filter.
- Logs a diagnostic message: how many swings were dropped, the threshold used, and the session p90 RMS.
- Called in `analyze_tennis_session()` immediately after `base_feats` is assembled from `swing_feat_df`, in both **medium mode** and **full mode**.

Bug 3 — Pre-warmup swings distort the fatigue trend plot

- The first `warmup_skip_min` minutes of swings (before the baseline period) had low MDF (82–101 Hz) due to poor electrode contact, giving +38–50 % drop % relative to the baseline. These swings were included in the plot, creating a misleading "V" shape (high drop % at start → 0 % at baseline → gradual increase).
- **Fix:** `_plot_fatigue_trend_stitched()` now accepts a `warmup_skip_ms` parameter. Swings with `t_peak_ms < warmup_skip_ms` are excluded from all four subplots. A green dotted vertical line marks the warmup boundary on every subplot. The plot title includes `| warmup=Nmin excluded` when pre-warmup swings are filtered.
- `analyze_tennis_session()` now computes `warmup_skip_ms` from `cfg.baseline.warmup_skip_min` and passes it to `_plot_fatigue_trend_stitched()`. Applied in both **medium mode** and **full mode**.

Expected behavior after fix

Condition	MDF drop %	Interpretation
Session start (fresh muscle, good electrode)	≈ 0 %	Baseline correctly anchored
Mid-session (no fatigue)	≈ 0 %	No spurious drift
Late session (genuine fatigue)	+12 % to +20 %	Mild → significant fatigue
Poor electrode contact swings	Excluded from baseline	No baseline corruption
Pre-warmup swings	Excluded from plot	No misleading V-shape at start

---

[2026-03-05] — Swing detection consistency: rename + cross-version tests

Changed (`emg_util_nopandas.py`)

- **Renamed `detect_swing_windows_from_imu_arrays` → `detect_swing_windows_from_imu`** to match the pandas version's function name in `emg_util.py`.
- Both files now expose the same canonical name for swing detection.
- Old name kept as a backward-compat alias: `detect_swing_windows_from_imu_arrays = detect_swing_windows_from_imu`
- Short alias also kept: `detect_swings_from_imu = detect_swing_windows_from_imu`
- Internal call in `stitch_decoded_session_from_paths()` updated to use the new name.

Verified: algorithm parity between `emg_util.py` and `emg_util_nopandas.py`

Both implementations of `detect_swing_windows_from_imu` are now confirmed identical in algorithm (only input type and FSR source differ by design):

Step	`emg_util.py` (pandas)	`emg_util_nopandas.py` (Android)
Time axis	`build_time_axis_from_ts(imu_df)`	`build_time_axis_from_ts(ts_ms_array)`
FSR source	DataFrame columns (`gyro_fsr_dps`, `acc_fsr_g`)	Function parameters
FSR maps	`_GYR_MAP`, `_ACC_MAP`	identical
`mag_dps`	`sqrt(gx²+gy²+gz²) / gyr_lsb`	identical
Acc gate	`amag_g >= thr_acc_g`	identical
Peak finding	contiguous above-threshold → `argmax`	identical
Min-gap	keep highest peak within `min_gap_ms`	identical
Window merge	merge overlapping `[t_start, t_end]`	identical
Output	`pd.DataFrame`	`List[Dict[str, float]]`

Added (`tests/test_regression_nopandas.py`)

- **`TestSwingDetectionCrossVersion`** — 6 new tests that run both the pandas (`emg_util.detect_swing_windows_from_imu_arrays`) and nopandas (`emg_util_nopandas.detect_swing_windows_from_imu`) implementations on identical synthetic IMU data and assert they produce the same results:
- `test_no_swings_both_empty` — no peaks above threshold → both return empty
- `test_single_swing_same_peak_time` — one peak → same `t_peak_ms` and `peak_mag`
- `test_two_swings_same_count_and_times` — two peaks > `min_gap_ms` apart → same count, times, boundaries
- `test_two_peaks_within_min_gap_keep_highest` — two peaks within `min_gap_ms` → both keep the higher peak
- `test_window_boundaries_identical` — `t_start = t_peak - pre_ms`, `t_end = t_peak + post_ms`
- `test_empty_input_both_return_empty` — empty arrays → both return empty

Updated (`tests/test_regression_nopandas.py`)

- `TestNoPandasSwingDetection.test_swing_detection_nopandas_from_arrays`: updated to call `nopandas.detect_swing_windows_from_imu()` (new canonical name).

Updated (`REGRESSION_TEST.md`)

- Status: `29/29 → 53/53 tests passing (100%) — 29 pandas + 24 nopandas`
- Added `TestSwingDetectionCrossVersion` to the nopandas test suite documentation.
- Added nopandas quick-start commands and performance table row.

[2026-03-05] — Nopandas regression tests: physical timeline + cross-platform parity

Added (`tests/test_regression_nopandas.py`) — 24 new tests

A new dedicated regression test file for the pandas-free Android pipeline. Tests use **synthetic CSV data** (no `frames.bin` required) and run in ~9 seconds.

`TestLoadDecodedCsvHostRxMs` (5 tests)

Verifies `host_rx_ms` is used as `ts_ms` when non-zero, falling back to `dev_us/1000` for legacy files (Layer 3 of the physical timeline fix).

`TestStitchDecodedSessionFromPaths` (7 tests)

Verifies `stitch_decoded_session_from_paths()` correctly handles:

- Single capture (baseline)
- Two captures with 10s BLE reconnect gap — duration preserved (~14s)
- ESP32 reset between captures (dev*us→0, host*rx_ms continuous) — no timestamp overlap
- Legacy files without `host_rx_ms` column — falls back to `dev_us/1000`
- Out-of-order paths — sorted by `ts_ms`, not filename order
- Empty path list — raises `FileNotFoundError`
- Two captures with IMU peaks — ≥2 swings detected, gap between swings preserved

`TestStitchDecodedSession` (2 tests)

Verifies `stitch_decoded_session(decoded_dir)` is a thin wrapper that produces identical results to `stitch_decoded_session_from_paths()`.

`TestComputeSwingEmgFeatures` (4 tests)

Verifies `compute_swing_emg_features()` output structure and IMU feature computation:

- All required keys present (`swing_id`, `mdf`, `rms`, `imu_mean_acc_dyn`, etc.)
- `imu_mean_acc_dyn` is finite and correct when IMU data provided
- `imu_mean_acc_dyn` is NaN when no IMU data
- Empty swing list → empty feature list

`TestBaselineAndFatigueDeltas` (6 tests)

Verifies baseline selection and fatigue delta computation:

- Baseline MDF = median of selected swings (~95.5 Hz)
- `mdf_drop_pct=20`, `rms_increase_pct=10`, `acc_drop_pct=20` computed correctly
- `mdf_drop_pct=0` when MDF equals baseline
- Warmup skip filters early swings from baseline
- Empty inputs return empty outputs

`TestNoPandasSwingDetection` (1 test, `@pytest.mark.slow`)

Requires real `frames.bin` in `regression_testlogs/`. Skipped automatically if absent.

[2026-03-03] — Feature: Add gyro_mag to live IMU subplot (`sensordata_plotter.py`)

Changed (`sensordata_plotter.py`)

- **IMU subplot (live mode) now shows both `acc_mag` and `gyro_mag`.**
- **Root cause of "slow variations":** `acc_mag` (total acceleration magnitude) always includes gravity (~1g at rest). The dynamic range visible on the 0–4g axis is only 1–3g, and the signal changes slowly because gravity dominates. `gyro_mag` (rotation rate in dps) is 0 at rest and spikes to 500–1000 dps during a tennis swing — it is the primary indicator of movement intensity.
- **Fix:** Added `gyro_buf = deque()` alongside `imu_buf`. Each `imu` packet now also appends `gmag` to `gyro_buf`. A secondary Y-axis (`ax_imu_gyro = ax_imu.twinx()`) plots `gyro_mag` in red (right axis, dps) while `acc_mag` remains in blue (left axis, g).
- **Auto-scaling:** `ax_imu_gyro` Y-axis starts at 200 dps minimum and expands to `max(gmag) × 1.1` each frame — so fast swings fill the subplot.
- **Combined legend:** both `acc_mag (g)` and `gyro_mag (dps)` shown in one legend box.
- **BLE reconnect:** `gyro_buf.clear()` added to the `seg=0` reset block.
- **`prune()`:** `gyro_buf` pruned with the same `imu_win` window as `imu_buf`.
- **`artists`:** `ln_gyro` added to the returned artists list.

[2026-03-03] — Fix: ENV_sw DC offset bug in `sensordata_plotter.py`

Fixed (`sensordata_plotter.py`)

- **`ENV_sw` (software envelope) was computing the DC level, not the true envelope.**
- **Root cause:** MyoWare RAW ADC values are unsigned and biased — the signal oscillates around ~1830–1843 (not the theoretical 2048). `abs(r)` always returns ~1843 (the DC level), so the Butterworth LPF converged to ~1843 at rest instead of ~0. The `ENV_sw` line appeared as a nearly flat line near the top of the plot, making the HW vs SW envelope comparison meaningless.
- **Fix (live mode):** Added `raw_dc_state` — a slow LPF (0.1 Hz) that tracks the RAW DC midpoint in real time. Initialized to 1843 (typical observed value) to minimize the ~10s convergence delay. Envelope is now `abs(r - raw_dc_state['lpf'])`.
- **Fix (offline mode):** Subtract the whole-file mean before computing the envelope: `raw_dc_offline = np.mean(raw)`, then `abs(x - raw_dc_offline)` for each sample. Instant — no convergence delay.
- **Before:** `env_filter.update(abs(r))` → LPF ≈ 1843 at rest (DC level, useless)
- **After:** `env_filter.update(abs(r - dc))` → LPF ≈ 0 at rest, ~40+ during contraction

Changed (`sensordata_plotter.py`)

- **Live mode** (`run_live()`):
- Added `raw_dc_state = {'lpf': 1843.0, 'alpha': None}` — DC tracker state dict
- `alpha` is computed once when `emg_sr_hz` is known: `1 - exp(-2π × 0.1 / sr_hz)`
- DC tracker updates on every `raw` packet: `lpf += alpha * (r - lpf)`
- `env_filter.update(abs(r - raw_dc_state['lpf']))` — correct envelope
- DC tracker reset to `{'lpf': 1843.0, 'alpha': None}` on new BLE session (`seg=0`)
- **Offline mode** (`run_offline()`):
- `raw_dc_offline = float(np.mean(raw))` — whole-file DC estimate
- `env_lpf = [env_filter.update(abs(x - raw_dc_offline)) for x in raw]`

Also fixed: ENV*sw scaled to match ENV*hw dynamic range (live + offline)

- **Root cause of "ENV*sw too small*":** *After DC removal, `abs(RAW-DC)` is the true AC amplitude (~0–100 counts), while ENV*hw (hardware output) has higher gain (~0–500 counts). The two signals are not directly comparable in absolute amplitude.
- **Shared helper:** `_compute_env_sw_scale(env_hw_max, env_sw_max)` — module-level function, single source of truth for the formula `max(ENV_hw) / max(ENV_sw)`. Returns 1.0 if ENV_sw is near zero (avoids division by zero at startup).
- **Fix (live mode):** `_env_sw_scale` (EMA, tau ≈ 20 frames) wraps `_compute_env_sw_scale()` called each frame with rolling-window maxima. ENV_sw is multiplied by this factor before plotting. Scale factor shown in subplot title: `ENV_sw (abs(RAW-DC)→LPF ×N.N)`.
- **Fix (offline mode):** `_compute_env_sw_scale()` called once with whole-file maxima. `env_lpf *= _env_sw_scale_off` before plotting. Scale factor shown in subplot title.
- **Interpretation:** The comparison is now qualitative (shape/timing match), not quantitative (absolute amplitude). Both signals should rise and fall together during muscle contractions.
- **Code unification:** The EMA is the only live-specific detail; the core formula `max(ENV_hw) / max(ENV_sw)` is identical in both modes via the shared helper.

Also fixed: ENV subplot Y-axis auto-scaling (live + offline)

- After the DC fix, `ENV_sw` values are in the 0–300 range while the Y-axis was fixed at 0–4095, making `ENV_sw` appear as a nearly invisible line near the bottom.
- **Live mode:** `_env_y_max` is computed each frame as `max(200, max(ENV_hw) × 1.25, max(ENV_sw) × 1.25)`. The ENV subplot Y-axis now auto-scales to show both signals clearly.
- **Offline mode:** `_env_off_max = max(200, max(ENV_hw) × 1.25, max(ENV_sw) × 1.25)`. Same logic applied once before `plt.show()`.
- The saturation (4000) and dead-signal (50) reference lines remain drawn; they will be off-screen when the signal is small, which is correct (no saturation present).

Expected behavior after fix

Condition	`ENV_sw` value	Visible on plot?
At rest (muscle relaxed)	~0–10 counts	Near bottom of plot ✓
Light contraction	~20–50 counts	Clearly visible ✓
Strong contraction	~100–300 counts	Tracks ENV_hw shape ✓
Saturation	~500+ counts	Matches ENV_hw saturation ✓

Note: same fix applied to Android `SessionCaptureActivity.java`

- Added `rawDcLpf = 1843f` + `rawDcAlpha = alphaForCutoff(0.1f, ENV_PLOT_HZ)`
- `decodeEmgBatch()`: `rawDcLpf += rawDcAlpha * (raw - rawDcLpf)` then `abs(raw - rawDcLpf)`
- `renderPlots()`: uses `rawDcLpf` as `rawCenter` for RAW zoom normalization

[2026-03-02] — Feature: Add dynamic RAW zoom & fix IMU scaling (`sensordata_plotter.py`)

Added (`sensordata_plotter.py`)

- Added `--raw-zoom` argument to optionally zoom the RAW subplot Y-axis.
- In **offline mode**, it zooms the Y-axis to `+/- 500` counts around the DC bias midpoint (`emg_ymax / 2.0`), which is ideal for inspecting low-gain resting-state noise.
- In **live mode**, it enables **dynamic Y-axis zooming** that automatically tracks the min/max of the rolling `raw_buf` (with a minimum padding of 100 counts to prevent extreme magnification of the baseline noise floor). This ensures the RAW signal optimally fills the screen regardless of the sensor gain setting or muscle effort.

Changed (`sensordata_plotter.py`)

- **IMU Subplot Scaling Fixed:** Removed dynamic zooming from the IMU (accelerometer) subplot. It now uses a fixed absolute minimum scale (0 to 4g) to preserve the visual sense of activity intensity. The axis will dynamically expand upwards only if extreme acceleration exceeds the 4g threshold, preventing data clipping while maintaining an absolute visual baseline.

[2026-03-02] — Fix: plotter stuck after BLE reconnect (`sensordata_plotter.py`)

Fixed (`sensordata_plotter.py`)

- **Plot frozen/stuck after BLE disconnect + reconnect.**
- **Root cause**: when `btCentral.py` reconnects it sends a new `emg_meta` packet with `seg=0`. The device's `dev_us` hardware timer resets to 0 on reconnect. The old code only reset the `FatigueMonitor` on `seg=0`, but left `t0_dev_us` pointing at the old (large) timestamp from

the previous session. Every new `dev_us` value was therefore a large negative number relative to `t0_dev_us`, so `prune()` immediately discarded every incoming sample and the rolling-window plot appeared frozen.
- **Fix** (inside the `emg_meta / seg=0` handler in `update()`):
  - `t0_dev_us = None` — forces re-anchor to the first new packet's timestamp.
  - `raw_buf`, `env_dev_buf`, `env_lpf_buf`, `imu_buf` cleared — removes stale pre-disconnect data so the rolling window starts fresh.
  - `recent_raw_dev_us` cleared — prevents sr-inference from mixing old/new timestamps.
  - `env_filter` re-created (if `emg_sr_hz` is known) — clears IIR state so the LPF does not carry stale history into the new session.
  - `in_peak` and `peak_gyro_mag` reset — fixes a pre-existing minor bug where a peak in progress at disconnect time would leave `in_peak=True` indefinitely.
- **No behaviour change for normal operation**: the guard `meta_seg == 0 and t0_dev_us is not None` is identical to the original; first connection (`t0_dev_us is None`) and segment rotations (`seg ≥ 1`) are unaffected. `swing_count`, `fatigue_result`, and `last_vitals` are intentionally NOT reset (session-level state that should survive a BLE reconnect).

[2026-02-25] — IMU FSR raised: gyro 250→1000 dps, acc 2→4 g (anti-saturation)

Changed (`myoware_imu_v2/myoware_imu_v2.ino`)

- `IMU_GYRO_FSR_DPS`: 250 → **1000** dps
- Tennis swings peak at ~500–1000 dps. FSR=250 saturated on every hard swing, clipping `gx/gy/gz` at ±32767 and making gyro-based swing detection unreliable.
- `IMU_ACC_FSR_G`: 2 → **4** g
- `imu_peak_acc_dyn` was showing median=2.464g (at the 2g ceiling). FSR=4g doubles the range while keeping 12-bit resolution at 0.122 mg/LSB.
- Both FSR codes are transmitted in every IMU meta frame (`FLAG_IMU_META_V2`), so the Python decoder applies the correct physical-unit scaling automatically. No decoder changes needed when changing these values.

Changed (fallback defaults — updated to match new firmware)

- `btCentral.py` lines 493-494: `cur_acc_fsr_g = 2 → 4`, `cur_gyro_fsr_dps = 250 → 1000`
- Used when a capture file has no FSR meta (old firmware or missing FLAG*IMU*META_V2).
- `emg_util/capture_loader.py` `load_imu_concat()` and `load_session_stitched()`: `acc_fsr_default_g=2.0 → 4.0`, `gyro_fsr_default_dps=250.0 → 1000.0`
- `emg_util/config.yaml`: `acc_fsr_default_g: 2 → 4`, `gyro_fsr_default_dps: 250 → 1000`
- `emg_util/config_fast.yaml`: same as above
- `emg_util/tennis_analysis_cloud.ipynb` Cell 5 `CLOUD_CONFIG`: `acc_fsr_default_g: 2 → 4`, `gyro_fsr_default_dps: 250 → 1000`

Note: Android `btcentral_offline_decode.py` — no change needed

- The FSR lookup tables (`ACC_FSR_CODE_TO_G`, `GYRO_FSR_CODE_TO_DPS`) are correct (they map all 4 FSR codes). The decoder reads FSR from the frame and writes it to CSV; no hardcoded defaults exist in the Android decoder.

[2026-02-25] — Per-swing IMU accelerometer features + acc*drop*pct fatigue indicator

Added (`emg_util_yaml_runner.py`)

- `per_swing_emg_features_stitched()` now accepts optional `imu_df` parameter. When provided, computes per-swing IMU accelerometer features:
- `imu_mean_acc_dyn` — mean `|acc_mag - 1g|` within swing window (effort/impact proxy)
- `imu_peak_acc_dyn` — peak `|acc_mag - 1g|` within swing window
- Note: gyro features omitted — gyro saturates at FSR=250 dps for tennis swings. Increase `gyro_fsr_dps` to 1000+ in firmware to unlock gyro-based features.
- `_plot_fatigue_trend_stitched()` now adds a 4th subplot (`acc_drop_pct`) when `swing_acc_drop_pct` is present in `swing_features.csv`.
- Teal scatter + rolling median trend line
- Reference lines: 0% (baseline), +10% (mild), +20% (significant)

Added (`emg_util.py`)

- `compute_baseline_from_swing_features()` now includes `imu_mean_acc_dyn` in the baseline dict (median of baseline swings).
- `apply_baseline_deltas()` now computes `{prefix}acc_drop_pct`:
- `acc_drop_pct > 0` means swing force decreased relative to baseline (fatigue signal)
- Only added when `imu_mean_acc_dyn` is present in both the DataFrame and baseline dict.

Changed (`analyze_tennis_session()`)

- Full mode now passes `imu_df` to `per_swing_emg_features_stitched()`.

Verified (log0207 session, 71 min, 6 BLE connections)

- `swing_features.csv`: 653 swings, all new columns present
- `imu_mean_acc_dyn`: median=0.869g (not saturated, reliable fatigue signal)
- `imu_peak_acc_dyn`: median=2.464g (saturated at 2g FSR — expected)
- `swing_acc_drop_pct`: median=+2.4% (slight force decrease over session)
- `baseline.json`: `imu_mean_acc_dyn=0.8991g` captured in baseline
- `fatigue_trend.png`: 4-subplot layout (MDF, RMS, MDF drop%, acc drop%)

[2026-02-25] — `preprocess_emg` single source of truth in `fatigue_monitor.py`

Changed (`fatigue_monitor.py`)

- Renamed `_preprocess_emg` → `preprocess_emg` (public API).
- `_preprocess_emg` kept as a private alias for internal use within the module.
- Updated docstring to document the 50 Hz low cutoff rationale and the public API contract.
- **`fatigue_monitor.py` is now the single source of truth for the EMG bandpass filter parameters** (50–450 Hz + 60/120 Hz notch). All callers import from here.

Changed (`emg_util.py`)

- Removed duplicate `preprocess_emg` function definition (~20 lines).
- Added `from fatigue_monitor import preprocess_emg` — all callers (`compute()`, `emg_util_yaml_runner.py`, etc.) now use the same filter as `FatigueMonitor.push_swing()`.

Changed (`emg_util_nopandas.py` — Android)

- Removed duplicate `preprocess_emg` function definition that still had the **old 20 Hz** low cutoff — it was never updated when `fatigue_monitor.py` was changed to 50 Hz.
- Added `preprocess_emg` to the existing `from fatigue_monitor import FatigueMonitor, preprocess_emg`.

Motivation

Three copies of `preprocess_emg` existed with diverging filter parameters:

- `fatigue_monitor.py`: 50 Hz (correct, updated)
- `emg_util.py`: 50 Hz (duplicate, now removed)
- `emg_util_nopandas.py`: **20 Hz** (stale, now removed)

The Android path was silently using a different filter than the PC path, producing inconsistent MDF/RMS features between offline PC analysis and on-device real-time monitoring. This is now fixed.

Maintenance rule: to change the EMG filter, edit only `emg_util/fatigue_monitor.py`. Android uses `emg_util` as a git submodule — changes are automatically available.

[2026-02-25] — EMG bandpass low cutoff raised from 20 Hz to 50 Hz

Changed (`fatigue_monitor.py`)

- `preprocess_emg`: low cutoff raised from **20 Hz** → **50 Hz**.

- Rationale: MyoWare 2.0 hardware bandpass is 20–500 Hz (confirmed from datasheet). The software 50 Hz cutoff additionally rejects the 20–50 Hz band (residual motion artifact + low-frequency EMG components less useful for MDF spectral estimation). This is a software choice beyond what the hardware provides, not a duplication of hardware filtering. Raises RMS accuracy and sharpens MDF estimates.
- Guard `if high <= 50.0: return y` updated accordingly.

[2026-02-25] — Extract FatigueMonitor to canonical `fatigue_monitor.py`

Added

- `emg_util/fatigue_monitor.py` — new self-contained canonical source for `FatigueMonitor`.
- Zero pandas / matplotlib dependency.
- Works on PC (full SciPy) and Android (Chaquopy, SciPy optional — numpy FFT fallback).
- Contains private helpers `_filtfilt_safe`, `_preprocess_emg`, `_mnf_mdf` (prefixed `_` to avoid namespace pollution).
- `FatigueMonitor` class with `reset()`, `push_swing()`, `get_summary()`.
- `btLeApp/app/src/main/python/fatigue_monitor.py` — copy of the above, deployed alongside `emg_util_nopandas.py` for Android/Chaquopy.

Changed (`emg_util.py`)

- Removed `FatigueMonitor` class definition (~160 lines).
- Added `from fatigue_monitor import FatigueMonitor` — re-exports the class so all existing callers (`sensordata_plotter.py`, etc.) continue to work unchanged.

Changed (`emg_util_nopandas.py`)

- Removed `FatigueMonitor` class definition (~160 lines).
- Added `from fatigue_monitor import FatigueMonitor` — re-exports the class so `analysis_bridge.py` continues to work unchanged.

Motivation

`FatigueMonitor` was duplicated in `emg_util.py` (PC) and `emg_util_nopandas.py` (Android) and required manual sync on every change. Extracting to `fatigue_monitor.py` gives a single source of truth. The only maintenance task going forward is keeping the Android copy of `fatigue_monitor.py` in sync with `emg_util/fatigue_monitor.py` (copy on change).

[2026-02-25] — Deduplication refactor: single source of truth in `emg_util.py`

Added (`emg_util.py`)

- `feat_mav(sig)` — Mean Absolute Value (classic EMG time-domain feature)
- `feat_rms(sig)` — Root Mean Square
- `feat_wl(sig)` — Waveform Length
- `feat_zc(sig, thr)` — Zero Crossings
- `feat_ssc(sig, thr)` — Slope Sign Changes
- `robust_zscore(x, baseline_mask, eps)` — Robust z-score using median + MAD; canonical implementation moved here from `emg_util_yaml_runner.py`
- `select_baseline_swings(swings, session_t0_ms, warmup_skip_min, warmup_skip_swings, baseline_swings)` — canonical baseline swing selection
- `compute_baseline_from_swing_features(swing_feats)` — compute robust median baseline from per-swing feature DataFrame
- `apply_baseline_deltas(df, base, prefix)` — add `mdf_drop_pct` and `rms_increase_pct` columns relative to baseline

Changed (`emg_util.py`)

- Removed `if __name__ == '__main__':` CLI entry point and `argparse` import. `emg_util.py` is now a **pure importable library**; the CLI lives exclusively in `emg_util_yaml_runner.py`.

Changed (`emg_util_yaml_runner.py`)

- Removed duplicate `robust_zscore()` function; `detect_swings_from_imu()` now calls `core.robust_zscore()`.
- `select_baseline_swings()` is now a thin wrapper that extracts YAML config params and delegates to `core.select_baseline_swings()`.
- `baseline_from_features()` is now a thin wrapper delegating to `core.compute_baseline_from_swing_features()`.
- `apply_deltas()` is now a thin wrapper delegating to `core.apply_baseline_deltas()`.
- Inline `mav / rms / wl / zc / ssc` computation blocks in `per_swing_emg_features()` replaced with `core.feat_mav/rms/wl/zc/ssc()` calls.
- Inline `mav / rms / wl / zc / ssc` computation blocks in `per_swing_emg_features_stitched()` replaced with `core.feat_mav/rms/wl/zc/ssc()` calls.

Motivation

All EMG feature math now has a single source of truth in `emg_util.py`.
Future fixes or tuning to `feat_zc`, `robust_zscore`, baseline logic, etc. only need to be made in one place.

[2026-02-25] — Tennis session stitching + medium mode

Added (`emg_util_yaml_runner.py`)

- `analyze_tennis_session(log_dir, cfg, out_dir)` — treats all BLE connections in a folder as one continuous tennis session; stitches EMG + IMU timelines using wall-clock `session_id` timestamps.
- `per_swing_emg_medium_stitched()` — fast per-swing features (envelope + MDF + RMS only); skips wl/zc/ssc/mnf/mav for speed.
- `windowed_emg_rms_stitched()` — fixed-window RMS/MAV across the full stitched session (O(N), no per-swing alignment).
- `_plot_fatigue_trend_stitched()` — MDF + RMS fatigue trend plot with BLE reconnection boundary markers.
- `_plot_windowed_rms()` — windowed RMS/MAV plot with reconnection markers.
- `_plot_imu_overlay_with_reconnections()` — IMU activity overlay with reconnection markers.
- `_save_tennis_session_summary()` — saves `session_summary.json` for the full tennis session.
- `features.mode == "medium"` — new analysis mode between `"bout"` (fast) and `"both"` (full).

Added (`capture_loader.py`)

- `load_session_stitched(sessions, ...)` — stitches multiple BLE capture sessions into one continuous `abs_us` timeline; returns `(emg_df, imu_df, reconnection_times_s, capture_ids)`.

Changed (`emg_util_yaml_runner.py`)

- `main()` now calls `analyze_tennis_session()` instead of the old per-BLE-connection loop.
- Old per-connection loop preserved as a comment for debugging reference.

[2026-02-25] — Activity gating + EMG health check

Added (`emg_util_yaml_runner.py`)

- `compute_hybrid_intensity(imu, cfg)` — hybrid gyro+dynamic-acc intensity signal with EMA smoothing.
- `activity_segments_from_imu(imu_df, cfg)` — percentile hysteresis segmentation of WORK PERIODS from IMU intensity.
- `swing_in_active_mask(swings, active_segs)` — boolean mask for swings within active segments.
- `plot_imu_activity_overlay(...)` — saves `imu_activity_overlay.png`.
- `segment_bouts_from_active(active_segs, swings, cfg)` — bout summary using active segments as primary structure.
- `_robust_z(x, ref_mask)` — internal robust z-score for activity gating (reference-mask variant; distinct from `robust_zscore`).

- `_ema_smooth(x, t_ms, tau_ms)` — time-aware exponential moving average.

Added (`capture_health.py`)

- `emg_bad_intervals_us(emg_paths, cfg)` — detects sporadic EMG disconnect/contact-loss intervals.
- `intervals_overlap(t0, t1, intervals)` — interval overlap test.

Changed (`emg_util_yaml_runner.py`)

- `analyze_single_session()` now runs EMG health check and optionally filters swings overlapping bad EMG intervals (`health.filter_swings_by_emg_bad`).

[2026-01-19] — Initial btCentral capture format support

Added

- `emg_util.py` — core EMG/IMU analysis library:
- `load_emg_imu_logs()`, `load_vitals_log()`
- `preprocess_emg()`, `envelope()`, `filtfilt_safe()`
- `mnf_mdf()`, `rms()`
- `compute()` — windowed feature extraction (RMS, MNF, MDF, IMU activity)
- `analyze_fatigue()` — MDF/MNF trend + onset detection
- `plot_all()`, `plot_paged()`
- `FatigueMonitor` — stateful real-time per-session fatigue tracker
- IMU detail helpers: `extract_imu_rows()`, `imu_segments()`, `build_time_axis_from_ts()`, `detect_movement_intervals()`, `run_imu_detail()`
- Phase-1 swing-aligned helpers: `detect_swing_windows_from_imu()`, `compute_emg_window_features()`, `compute_swing_emg_features()`
- `emg_util_yaml_runner.py` — YAML-driven CLI runner
- `capture_loader.py` — session discovery and data loading
- `config.yaml`, `config_fast.yaml` — analysis configuration
- `capture_loader.py` — `discover_sessions()`, `load_emg_concat()`, `load_imu_concat()`, `summarize_session()`, `make_interleaved_df()`